BAB
II
OPERASI DASAR PADA CITRA DIGITAL

Materi

- Operasi Dasar pada Citra
- Operasi Aritmatika pada Citra

Tujuan Pembelajaran

- Dengan mempelajari bab ini Mahasiswa dapat mengetahui dan mampu menerapkan operasi-operasi dasar dan operasi aritmatika pada citra digital.

A. Penyajian

1. Operasi Dasar pada Citra

Ada empat operasi dasar dalam pengolahan citra digital yaitu 1) mengakses dan memodifikasi piksel, 2) mengakses properti pada citra, 3) menetapkan *Region of Interest* (ROI), dan 4) memecah (*splitting*) dan menggabungkan (*merging*) citra [1]. Berikut ini penjelasan dari keempat operasi dasar pada citra digital.

a. Mengakses nilai dan memodifikasi piksel

Citra disusun oleh piksel-piksel dengan satu atau lebih layer atau *channel*. Piksel yang dalam Bahasa Inggris disebut *pixel* (*picture element*) adalah unsur gambar atau representasi sebuah titik terkecil dalam sebuah gambar grafis [2]. Operasi paling dasar yang akan sering dilakukan adalah mendapatkan nilai dan memodifikasi nilai pada posisi piksel dan *channel* tertentu. Anda dapat mengakses nilai piksel dengan koordinat baris dan kolomnya. Untuk gambar BGR (tiga *channels*), mengembalikan nilai matriks biru (B), hijau (G),

Dipindai dengan CamScanner

dan merah (R). Untuk citra grayscale (satu *channel*), h
mengembalikan nilai intensitas yang sesuai.

Berikut ini merupakan fungsi pada OpenCV untuk menga
nilai pada piksel atau koordinat x,y. fungsi di bawah ini ar
variabel pixelVal akan menampung nilai dari matriks citra di p
atau koordinat x,y (jika mode RGB/BGR akan terdiri dari 3 nila

```
pixelVal = img[x,y]
```

Berikut ini merupakan *code* untuk memodifikasi nilai pada p
atau koordinat x,y. *Code* di bawah ini artinya matriks citra
koordinat x,y akan memiliki nilai 255 untuk setiap *channel*
(B=255, G=255, dan R=255).

```
img[x,y] = [255,255,255]
```

b. Mengakses properti citra

Citra yang telah dibaca menggunakan fungsi pada Oper
memiliki properti yang dapat kita akses untuk kebutuhan terte
Properti pada citra antara lain:

- Bentuk citra : Ukuran baris, kolom, dan *channel* diak
  menggunakan *script* **img.shape**
- Ukuran citra : Total jumlah piksel diakses menggunakan s
  img.size.
- Tipe data citra: Tipe data dari citra yang dapat diak
  menggunakan *script* **img.dtype**

c. Menetapkan *Region of Interest* (ROI)

Terkadang, Anda harus bermain dengan wilayah gambar terte
Misalnya, untuk deteksi mata dalam gambar dan deteksi wajah
keseluruhan citra. Ketika wajah diperoleh, kami memilih wila
wajah saja dan mencari mata di dalamnya daripada mencari sel
gambar. Ini meningkatkan akurasi dan kinerja.

Dipindai dengan CamScanner

ROI diperoleh dengan menggunakan *library* Numpy. Berikut ini merupakan kode yang digunakan untuk menerapkan ROI.

```
imgROI = img[x1:x2, y1:y2]
```

d. Memecah (*Splitting*) dan Menggabungkan (*Merging*) Citra

Terkadang Anda perlu bekerja secara terpisah pada setiap *channel* gambar B, G, R. Dalam hal ini, Anda perlu membagi gambar BGR menjadi satu saluran. Dalam kasus lain, Anda mungkin perlu menggabungkan masing-masing saluran ini ke gambar BGR.

Untuk memecah (*splitting*) gambar Anda dapat melakukannya hanya dengan:

```
b,g,r = cv.split(img)
```

atau:

```
b = img[:,:,0]
g = img [:,:,1]
r = img [:,:,2]
```

Untuk menggabungkan (*merge*) gambar Anda dapat melakukannya hanya dengan:

```
imgMerge = cv.merge((b,g,r))
```

Untuk menjadikan semua piksel di *channel* merah ke nol, Anda tidak perlu memecah *channel* terlebih dahulu. Pengindeksan dengan menggunakan numpy akan lebih cepat dan mudah.

Dipindai dengan CamScanner

```
imgRgb[:,:,2] = 0
```

2. **Operasi Aritmatika pada Citra**

a. *Image Addition*

Anda dapat menambahkan dua gambar dengan fungsi Ope cv.add () atau hanya dengan operasi numpy, res = img1 + i Kedua gambar harus memiliki kedalaman (*depth*) dan tipe gambar yang sama, atau gambar kedua hanya bisa menjadi skalar. Ada perbedaan antara *addition* OpenCV dan penamb Numpy. Penambahan OpenCV adalah operasi jenuh seme penambahan Numpy adalah operasi modulo. *Image Addition* digunakan untuk menambah tingkat kecerahan pada suatu citra citra ditambahkan dengan suatu nilai scalar.

Image addition dengan menggunakan openCV:

```
res = cv.add(img1, img2)
```

Image addition dengan menggunakan operasi numpy:

```
res = img1 + img2
```

b. *Image Subtraction*

Sama seperti penambahan pada gambar, *image subtraction* dapat dilakukan dengan openCV dan numpy. *Image subtrac* juga digunakan untuk mengurangi tingkat kecerahan pada suatu jika citra ditambahkan dengan suatu nilai scalar.

Image subtraction dengan menggunakan openCV:

```
res = cv.subtract (img1, img2)
```


Dipindai dengan CamScanner

Image subtraction dengan menggunakan operasi numpy:

```
res = img1 + img2
```

c. *Bitwise Operations*

Operasi bitwise pada citra adalah melakukan pengoperasian satu atau lebih gambar menggunakan operator logika pada citra. Operasi bitwise antara lain AND, OR, NOT, dan XOR.

Operasi bitwise	Code OpenCV
AND	dst = cv.bitwise_and(src1, src2, mask)
OR	dst = cv.bitwise_not(src1, src2, mask)
NOT	dst = cv.bitwise_or(src1, src2, mask)
XOR	dst = cv.bitwise_xor(src1, src2, mask)

Keterangan:

src1 : input array pertama atau skalar.
src2 : input array kedua atau skalar.
mask : opsional, array saluran tunggal 8-bit, yang menentukan elemen array output yang akan diubah.

B. Latihan

1. Membuat program dengan OpenCV-Python untuk melihat perbedaan properti dari citra BGR/RGB dengan citra *grayscale*.

```
import numpy as np
import cv2 as cv

imgRgb = cv.imread('upn.jpg')
imgGrayscale = cv.imread('upn.jpg', cv.IMREAD_GRAYSCALE)

print('\nRGB Image')
print ( imgRgb.shape )
print( imgRgb.size )
print( imgRgb.dtype )
```


Dipindai dengan CamScanner

```
print('\nGrayscale Image')
print ( imgGrayscale.shape )
print( imgGrayscale.size )
print( imgGrayscale.dtype )
```

Keluaran:

RGB Image	Grayscale Image
(284, 500, 3)	(284, 500)
426000	142000
uint8	uint8

Keterangan:

Bentuk (*shape*) citra BGR/RGB keluarannya 3 nilai yaitu (28
3) di mana 284 adalah ukuran tinggi/baris dari citra, 500
ukuran lebar/kolom dari citra, dan 3 adalah jumlah channe
layer dari citra. Sedangkan bentuk atau shape dari citra gra
keluarannya adalah 2 nilai yaitu (284, 500), tidak ada ju
channel atau layer karena jumlah *channel* pada citra *gra*
hanya 1 *channel* saja. Size dari citra BGR juga berbeda dari
grayscale, di mana untuk citra BGR adalah 426000 (284 x 500
dan citra grayscale adalah 142000 (284 x 500). Untuk tipe
kedua citra tersebut adalah sama yaitu bertipe data integer de
nilai perpikselnya adalah 8 bit (0-255).

Dipindai dengan CamScanner

2. Buatlah program dengan OpenCv-Python untuk mengakses dan memodifikasi piksel serta *split* dan *merge* citra.

```
import numpy as np
import cv2 as cv

imgRgb = cv.imread('upn.jpg')
imgGrayscale = cv.imread('upn.jpg', cv.IMREAD_GRAYSCALE)

pxRgb = imgRgb[100,100]
blue = imgRgb[100,100,0]
imgRgb[100,100] = [255,255,255]
pxGrayscale = imgGrayscale[100,100]
imgGrayscale[100,100] = 255

print(pxRgb)
print(blue)
print(imgRgb[100,100])
print(pxGrayscale)
print(imgGrayscale[100,100])

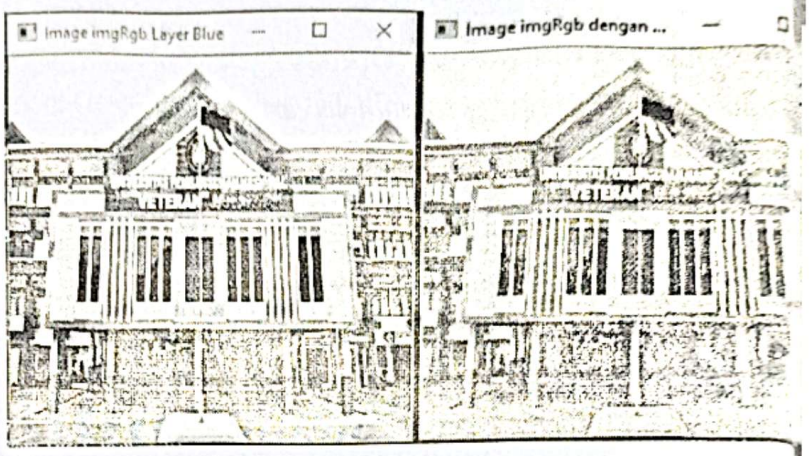
b,g,r = cv.split(imgRgb)
imgRgbMerge = cv.merge((b,g,r))
b = imgRgb[:,:,0]
imgRgb[:,:,2] = 0

cv.namedWindow('Image imgRgb Layer Blue', cv.WINDOW_NORMAL)
cv.imshow('Image imgRgb Layer Blue', b)
cv.namedWindow('Image imgRgb dengan Layer Red=0',
cv.WINDOW_NORMAL)
cv.imshow('Image imgRgb dengan Layer Red=0', imgRgb)
cv.waitKey(0)
cv.destroyAllWindows()
```

Keluaran:

[255 255 255]
111
[255 255 255]
125
255

Dipindai dengan CamScanner



Gambar 7. Output Citra pada Latihan No. 2.

Keterangan:

- pxRgb = imgRgb[100,100] yaitu mengakses matriks imgRgb pada posisi koordinat piksel 100,100 di semua ch (B, G, dan R).
- blue = imgRgb[100,100,0] yaitu mengakses matriks imgRgb pada posisi koordinat piksel 100,100 hanya di ch blue saja.
- imgRgb[100,100] = [255,255,255] yaitu mengubah nilai m citra imgRgb pada posisi koordinat piksel 100,100 untuk ch B=255, G=255, dan R=255.
- b = imgRgb[:,:,0] yaitu mengakses matriks citra imgR semua baris dan semua kolom pada channel ke-0 (channel b
- imgRgb[:,:,2] = 0 yaitu memberikan nilai 0 untuk matriks imgRgb di semua baris dan semua kolom pada channel (channel *red*).

Dipindai dengan CamScanner

3. Lakukan penduplikasian gambar bendera pada citra upn.jpg sedemikian sehingga bendera terlihat menjadi 2 pada posisi yang berbeda.

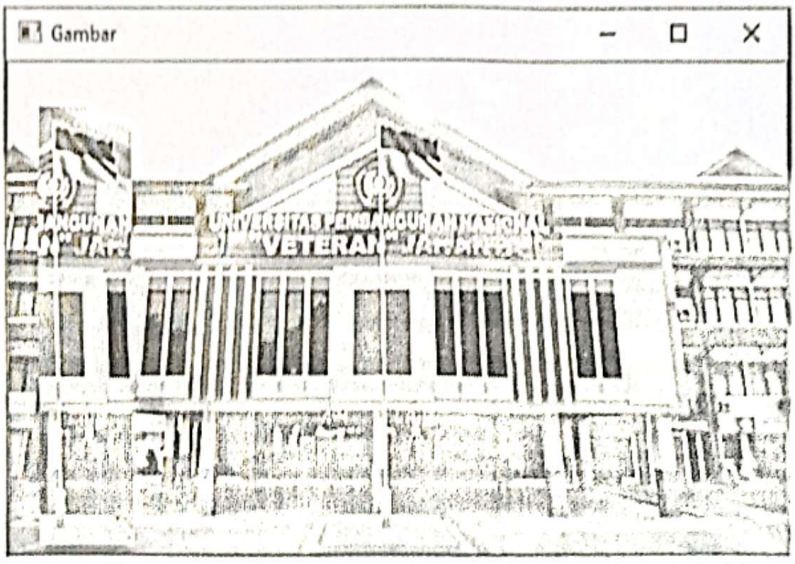
```
import numpy as np
import cv2 as cv

imgRgb = cv.imread('upn.jpg')

tiang = imgRgb[25:280, 230:290]
imgRgb[25:280, 20:80] = tiang

cv.imshow('Gambar', imgRgb)
cv.waitKey(0)
cv.destroyAllWindows()
```

Keluaran:



Gambar 8. Output Citra pada Latihan No. 3.

Keterangan:

- Untuk kode tiang = imgRgb[25:280, 230:290] artinya mengambil region of interest citra imgRGB di posisi baris 25-280 dan posisi kolom 230-290. Nilai [25:280, 230:290] diperoleh dari trial and

Dipindai dengan CamScanner

error atau coba-coba sampai mendapatkan posisi tiang be
yang tepat.

- Selanjutkan kopikan tiang bendera tersebut ke posisi lain de
kode imgRgb[25:280, 20:80] = tiang. Nilai [25:280, 20:80]
diperoleh dari trial and error.

4. Membuat program untuk menggabungkan dua buah citra. Citra
digunakan adalah logo UPN Veteran Jakarta dan foto Kampus
Veteran Jakarta.

```
import numpy as np
import cv2 as cv

imgUpn = cv.imread('upn.jpg')
imgUpnLogo = cv.imread('LogoUPNVJ.png')

# meletakkan logo di sudut kiri atas, dan menyimpannya membuat R
rows,cols,channels = imgUpnLogo.shape
roi = imgUpn[0:rows, 0:cols ]

# Buat sebuah mask dari logo dan mask inverse logo
img2gray = cv.cvtColor(imgUpnLogo,cv.COLOR_BGR2GRAY)
ret, mask = cv.threshold(img2gray, 10, 255, cv.THRESH_BINARY)
mask_inv = cv.bitwise_not(mask)

# area black-out dari logo dalam ROI
img1_bg = cv.bitwise_and(roi,roi,mask = mask_inv)

# ambil hanya region dari logo dalam gambar logo.
img2_fg = cv.bitwise_and(imgUpnLogo,imgUpnLogo,mask = mask)

# letakan logo dalam ROI dan modifikasi citra utama
dst = cv.add(img1_bg, img2_fg)
imgUpn[0:rows, 0:cols ] = dst

cv.imshow('Logo UPNVJ', imgUpn)
cv.waitKey(0)
cv.destroyAllWindows()
```


Dipindai dengan CamScanner

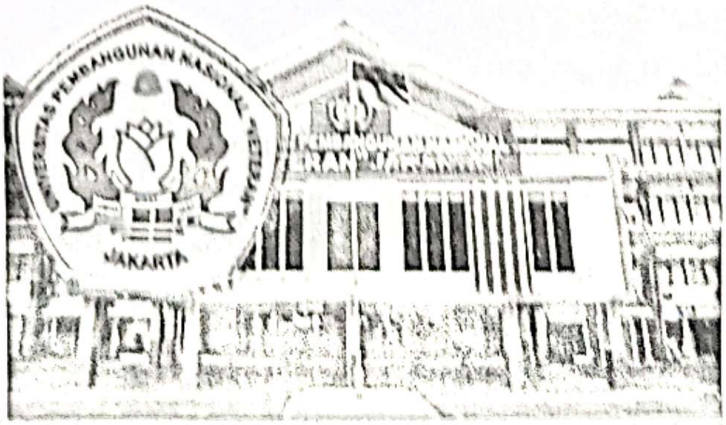
Keluaran:



Gambar 9. LogoUPNVJ.png.



Gambar 10. upn.jpg.



Gambar 11. Hasil Penggabungan.

Dipindai dengan CamScanner

Keluaran:



Gambar 9. LogoUPNVJ.png.



Gambar 10. upn.jpg.



Gambar 11. Hasil Penggabungan.

Dipindai dengan CamScanner

C. Lembar Kerja Praktikum

1. Baca 2 buah gambar dengan format *.jpg, yaitu foto Pr pertama Indonesia Ir. Soekarno dan Wakil Presiden pe Indonesia Dr. Drs. H. Mohammad Hatta. Ubah mode kedua g tersebut menjadi *grayscale* dan tampung ke dalam 2 variabel berbeda.



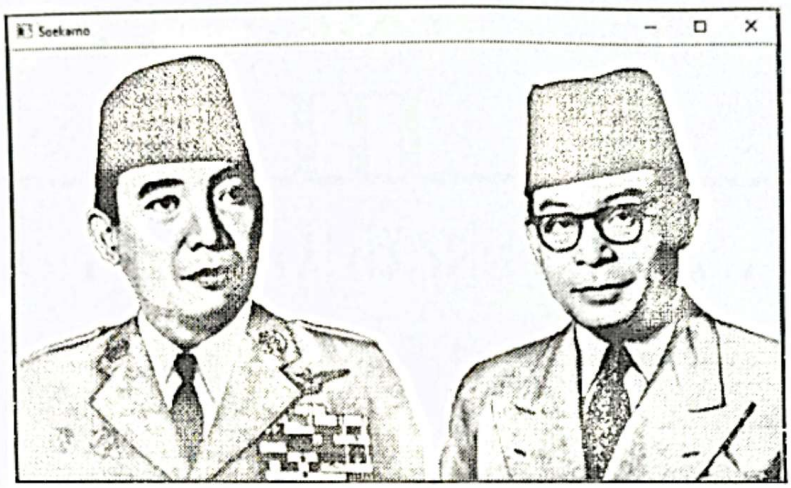
Gambar 12. Soekarno.jpg.



Gambar 13. Hatta.jpg.

2. Jika kedua memiliki ukuran yang berbeda, maka sesuaikan uk kedua gambar tersebut sedemikian sehingga memiliki ukuran p yang sama, yaitu dengan cara mengambil *shape* (*rows* dan *cols*) gambar yang lebih kecil dan dapatkan ROI gambar yang lebih b sesuai dengan shape gambar yang lebih kecil.
3. Lakukan operasi terhadap kedua gambar tersebut (yang ukura sudah sama atau sesuai):
   a. *Image Addition*
   b. *Image Subtraction*
4. Tampilkan empat citra:
   a. Citra *grayscale* pertama
   b. Citra *grayscale* kedua
   c. Citra hasil *image addition*
   d. Citra hasil *image subtraction*
5. Jelaskan hasil dari operasi *Image Addition* dan *Image Subtractio*
6. Buatlah program dengan OpenCV-Python untuk menggabung kedua gambar Ir. Soekarno dan Dr. Drs. H. Mohammad H sedemikian sehingga gambar menjadi sebagai berikut:

Dipindai dengan CamScanner



Gambar 14. Keluaran yang Diharapkan dari Penggabungan Kedua Citra.

Referensi

[1] A. Mordvintsev and A. K, "OpenCV-Python Tutorials Documentation." p. 265, 2014.

[2] Wikipedia, "Piksel," 2019. https://id.wikipedia.org/wiki/Piksel.

Dipindai dengan CamScanner

BAB

III

RUANG WARNA PENGOLAHAN CITRA DIGI

Materi

- Konsep Ruang Warna
- Model Warna
- Transformasi Ruang Warna pada Citra Digital

Tujuan Pembelajaran

- Mahasiswa dapat menjelaskan konsep ruang warna pada pengol citra digital.
- Mahasiswa mampu menyebutkan dan menjelaskan model w pada pengolahan citra digital.
- Mahasiswa mampu melakukan transformasi warna pada pengol citra digital menggunakan Python-openCV.

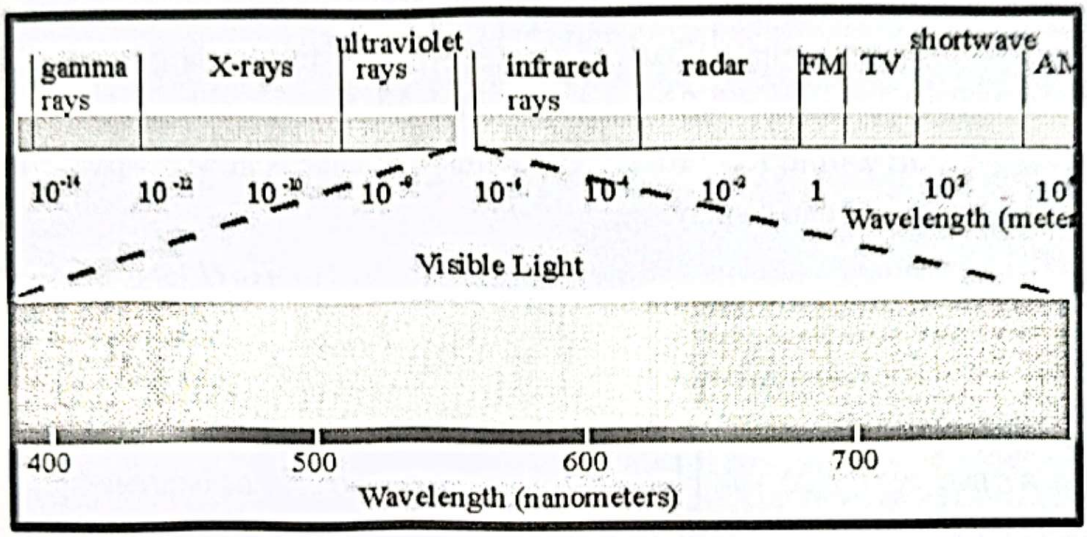
A. Penyajian

1. Konsep Warna

Pada dasarnya, warna pada objek yang dilihat manusia ditentu oleh sifat cahaya yang dipantulkan dari objek tersebut. Seperti y diilustrasikan pada Gambar 15, cahaya tampak terdiri dari pita freku yang relatif sempit dalam spektrum elektromagnetik, yaitu berkisar an 380-780 nanometer. Secara teknik, warna adalah spektrum tertentu y terdapat di dalam suatu cahaya sempurna (berwarna putih). Benda y mendukung pemantulan dalam rentang spektrum yang terli menunjukkan beberapa corak warna. Identitas suatu warna ditentu panjang gelombang cahaya tersebut. Misalnya, objek berwarna hij memantulkan cahaya dengan panjang gelombang terutama dalam kisa

Dipindai dengan CamScanner

500 hingga 570 nm, sedangkan warna biru memiliki panjang gelombang 460 nm. [1].



Gambar 15. Spektrum Warna

Cahaya yang kita lihat adalah bukan cahaya dengan satu panjang gelombang melainkan kumpulan panjang gelombang tertentu. Warna terbentuk dari kumpulan gelombang dengan panjang gelombang yang berbeda-beda. Ini bisa diartikan bahwa warna yang kita lihat adalah kombinasi dari beberapa elemen dasar warna.

2. Model Warna

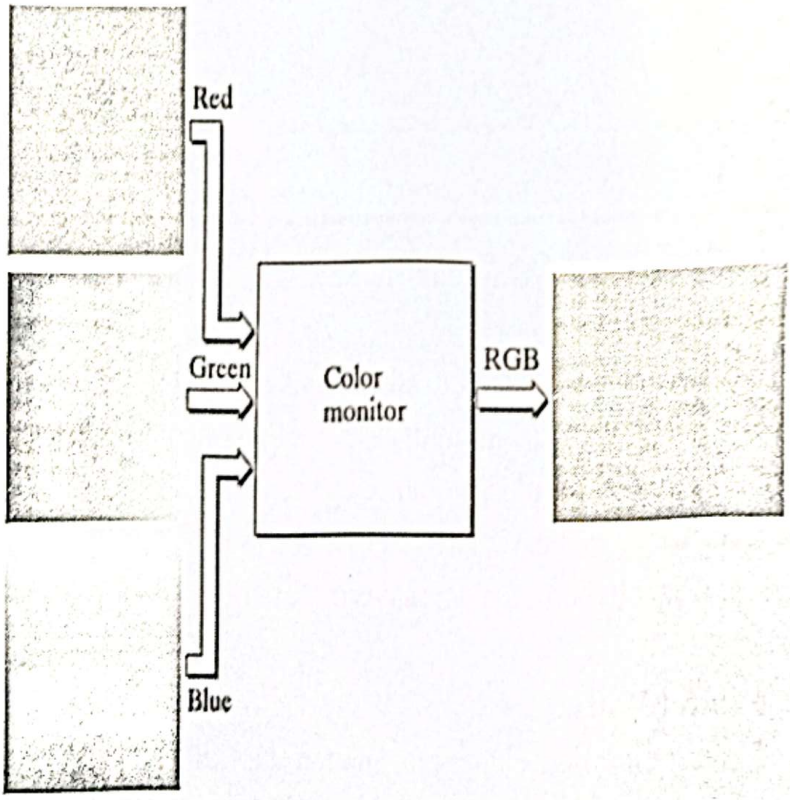
Cara penyajian campuran elemen dasar untuk menghasilkan warna dinamakan dengan Color Space atau Ruang Warna. Beberapa macam ruang warna dalam pengolahan citra digital, dan yang paling umum ditemui adalah ruang warna RGB, yaitu model ruang warna yang memiliki 3 channel yaitu red, green, blue. Masih ada beberapa lagi ruang warna yang dapat kita temui dalam pengolahan citra digital, di antaranya grayscale, CMYK, HSV, YCbCr. Berikut ini penjelasannya:

a. Model Warna RGB/BGR

Gambar pada model warna RGB terdiri dari tiga komponen gambar merah (*red*), hijau (*green*), dan biru (*blue*). Pada monitor yang menggunakan model warna RGB, ketiga gambar ini digabungkan untuk menghasilkan gambar warna komposit, dapat dilihat pada

Dipindai dengan CamScanner

gambar di bawah ini. Jumlah bit yang digunakan untuk me
setiap piksel dalam ruang RGB disebut kedalaman piksel (
depth). Pada citra RGB di masing-masing channel merah, hija
biru menampung 8-bit per-piksel. Maka setiap titik pada g
akan menampung 3x8 bit, atau 24 bit per-titik pada gambar
Istilah gambar full-color ini sering disebut sebagai gambar
RGB 24-bit. RGB digunakan untuk menghasilkan warna pada
monitor dan display.



Gambar 16. CRT *Display*.

b. Model Warna CMY/CMYK

Model warna CMYK (cyan, magenta, yellow, dan key) ad
proses pencampuran pigmen yang biasanya digunakan percet
Tinta elemen cyan, magenta, yellow, dan lebih lengkapnya dita
elemen black. Keempat elemen tersebut dicampurkan de
komposisi tertentu dan akurat sehingga menghasilkan warna t
seperti yang diinginkan. Sistem CMY(K) juga digunakan
banyak printer warna dan mesin fotokopi. Maka dari itu diper

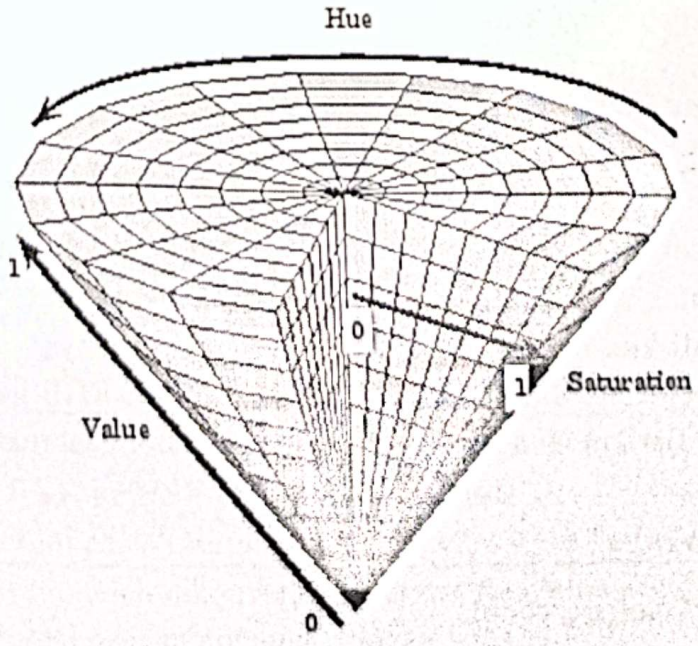
Dipindai dengan CamScanner

input data CMY(K) atau melakukan konversi dari RGB ke CMY(K). Konversi ini dilakukan dengan menggunakan operasi sederhana.

$$\begin{bmatrix} C \\ M \\ Y \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} - \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

c. Model Warna HSV

Ruang warna HSV (*Hue*, *Saturation*, *Value*). *Hue* merupakan suatu nilai yang merepresentasikan spektrum warna dari cahaya tampak (merah, jingga, kuning, hijau, biru, dan ungu). *Saturation* merupakan nilai yang menunjukkan tingkat kejenuhan atau kemurnian dari suatu warna. Semakin besar nilai saturasi maka semakin murni warna yang dihasilkan. Sedangkan *value* dapat didefinisikan sebagai nilai yang menunjukkan tingkat kecerahan warna. HSV dikembangkan menggunakan sistem koordinat polar dan banyak digunakan untuk fitur warna pada gambar. Gambar 17 adalah representasi ruang warna pada HSV.

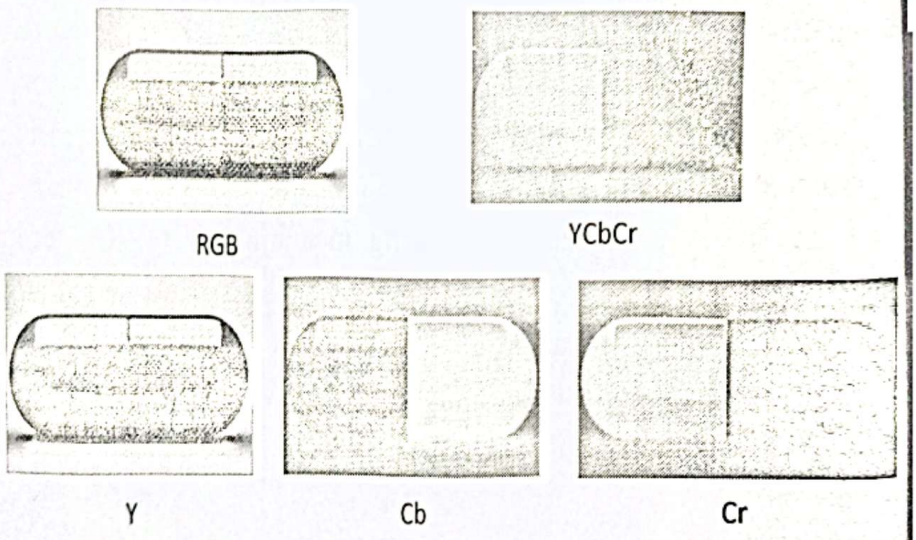


Gambar 17. Representasi Ruang Warna pada HSV.

Dipindai dengan CamScanner

d. Model Warna YCbCr

Komponen penyusun citra YCbCR antara lain Y mer komponen luma sedangkan Cb dan Cr merupakan komponen perbedaan antara biru dan merah. Pada Gambar 18, menam bahwa gambar Y merupakan citra skala abu-abu (greyscale gambar aslinya. Pada gambar, salju ditampilkan dengan nilai sedang (tidak rendah dan tidak tinggi) antara Cr dan Cb; g ditampilkan oleh Cb yang lemah dan Cr yang kuat; rumput ditampilkan oleh Cb dan Cr yang sama-sama lemah; serta lang ditampilkan oleh Cb kuat dengan Cr lemah.



Gambar 18. Citra RGB dan Citra YCbCr dari Setiap Channel-nya

3. Transformasi Ruang Warna pada Citra Digital

Transformasi ruang warna adalah proses mengubah ruang w satu ke bentuk ruang warna lainnya. Dalam pengolahan citra digital sering sekali kita melakukan transformasi ini, misalnya, mengubah RGB ke grayscale, RGB ke HSV atau sebaliknya, RGB ke YCbCR sebaliknya. Berikut ini adalah fungsi untuk transformasi ruang warna p citra digital:

```
dst=cv.cvtColor(src, code)
```

Dipindai dengan CamScanner

Keterangan:

- dst : *output* citra dengan size dan kedalaman yang sama dengan citra input
- src : citra input
- code : konversi ruang warna
  Dalam OpenCV Ada lebih dari 150 metode atau konversi kode ruang warna yang tersedia di OpenCV, dapat dilihat pada link:
  https://docs.opencv.org/4.1.2/d8/d01/group__imgproc__color__conversions.html

a. RGB/BGR ke *Grayscale*

Transformasi ruang RGB adalah konversi ke/dari warna RGB 16-bit (R5: G6: B5 atau R5: G5: B5), serta konversi ke/dari skala abu-abu menggunakan:

$$\text{RGB[A] to Gray:} \quad Y \leftarrow 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$$

$$\text{Gray to RGB[A]:} \quad R \leftarrow Y, G \leftarrow Y, B \leftarrow Y, A \leftarrow \max(ChannelRange)$$

Berikut ini kode konversi ruang warna yang dapat digunakan untuk transformasi ruang warna RGB/BGR ke *grayscale* atau sebaliknya [2].

```
cv.COLOR_BGR2GRAY
cv.COLOR_RGB2GRAY
cv.COLOR_GRAY2BGR
cv.COLOR_GRAY2RGB
```

b. RGB/BGR ke HSV dan HSV ke RGB/BGR

Berikut ini adalah formula konversi citra RGB ke HSV dengan citra RGB yang telah dikonversi ke format floating-point dan diskalakan agar sesuai dengan rentang 0 hingga 1.

Dipindai dengan CamScanner

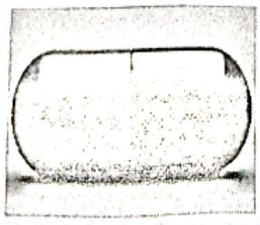
```
#menyimpan file citra hasil transformasi
cv.imwrite('Capsule Grayscale.jpg',imgGrayscale)
cv.imwrite('Capsule HSV.jpg',imgHSV)
cv.imwrite('Capsule YCbCr.jpg',imgYCrCb)

cv.imshow('RGB',imgCr)
cv.imshow('Grayscale',imgGrayscale)
cv.imshow('HSV',imgHSV)
cv.imshow('YCbCr',imgYCrCb)
cv.imshow('Y',imgY)
cv.imshow('Cb',imgCb)
cv.imshow('Cr',imgCr)
cv.waitKey(0)
cv.destroyAllWindows()
```

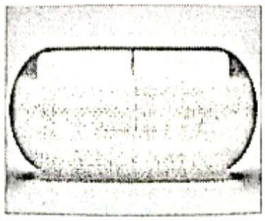
Keluaran Console:

Shape RGB : (800, 1480, 3)
Shape Grayscale : (800, 1480)
Shape HSV : (800, 1480, 3)
Shape YcbCr : (800, 1480, 3)

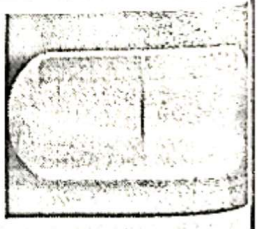
Keluaran Citra:



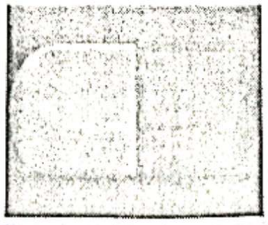
RGB



Grayscale

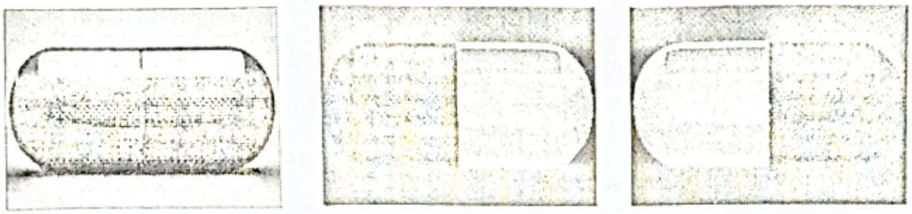


HSV



YCbCr

Dipindai dengan CamScanner



Gambar 19. Output Citra Latihan No. 1.

2. Setelah kita mengetahui bagaimana mengkonversi gambar BGR/RGB ke *grayscale*, HSV, YCbCr/YCrCb, kita dapat menggunakannya untuk mengekstrak objek berwarna. Dalam YCbCr, lebih mudah untuk mewakili warna daripada di ruang warna BGR. Dalam aplikasi, Kita akan mencoba mengekstraksi objek apel berwarna merah. Jadi inilah caranya:
   - Baca citra dengan nama file 'red_apple.jpg'.
   - Konversi citra dari BGR ke ruang warna YCbCr/YCrCb.
   - Kami memberi ambang batas (*threshold*) gambar YCbCr/YCrCb untuk rentang warna merah.
   - Ekstrak objek merah saja, kita bisa melakukan apa saja pada gambar yang kita inginkan.
   - Di bawah ini adalah kode yang dikomentari secara rinci:

```
import numpy as np
import cv2 as cv

#membaca citra RGB
rgb = cv.imread('red_apple.jpg')
ycrcb = cv.cvtColor(rgb, cv.COLOR_BGR2YCrCb)

lower_blue = np.array([40,150,50])
upper_blue = np.array([200,255,150])

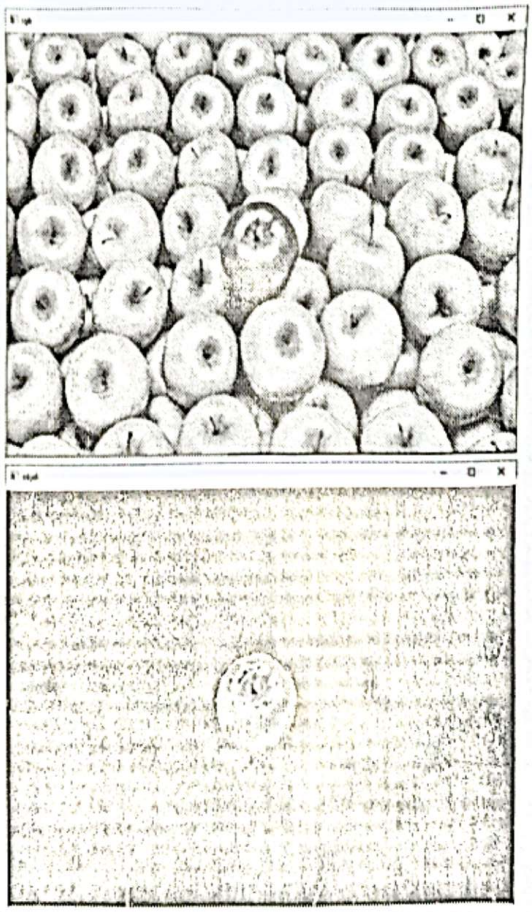
# Threshold the HSV image to get only red colors
mask = cv.inRange(ycrcb, lower_blue, upper_blue)
```


Dipindai dengan CamScanner

```
res = cv.bitwise_and(rgb,rgb, mask= mask)

cv.imshow('rgb', res)
cv.waitKey(0)
cv.destroyAllWindows()
```

Keluaran:



Gambar 20. Output Citra Latihan No. 2.

C. Lembar Kerja Praktikum

1. Lakukan analisa singkat dari hasil program yang telah dibuat latihan nomor 2.
2. Lakukan akuisisi citra menggunakan kamera dengan objek b dalam satu citra terdapat dua objek (satu objek dominan warna dan satu objek lainnya dominan warna merah/coklat). Kedua tersebut diakuisisi di atas kertas putih. Lakukan segmentasi d

Dipindai dengan CamScanner

melakukan konversi ruang warna terlebih dahulu dari ruangan warna RGB/BGR ke YCbCr, dan hasil pemrosesan citra tersebut menghasilkan dua *output* citra dengan ketentuan sebagai berikut:

a. *Output* citra pertama menghasilkan citra dengan latar belakang (*background*) citra menjadi hitam (atau bernilai 0) dan **objek citra dominan warna biru** yang kalian ambil menjadi warna sebenarnya (rgb *true color*).
b. *Output* citra kedua menghasilkan citra dengan latar belakang (*background*) citra menjadi hitam (atau bernilai 0) dan **objek citra dominan warna merah/coklat** yang kalian ambil menjadi warna sebenarnya (rgb *true color*).

Referensi

[1] R. C. Gonzalez and R. E. Woods, "Digital Image Processing, 4th Edition," *J. Electron. Imaging*, p. 1019, Jan. 2018.

[2] A. Mordvintsev and A. K, "OpenCV-Python Tutorials," 2019. https://docs.opencv.org/4.1.2/d6/d00/tutorial_py_root.html.

Dipindai dengan CamScanner